

Assignment 1

DUE DATE: 11:59PM (MELBOURNE TIME), MONDAY 30TH MARCH, 2026

This assignment is worth 10% of your total mark.

You will work in pairs for the assignment. Each pair will submit only one solution, produced jointly by both partners. Working in pairs is important since a significant part of the assignment is brainstorming security threats to a system, using the STRIDE methodology discussed in lectures. As with other brainstorming activities, security threat enumeration is an inherently creative process that will benefit from being performed by a pair, rather than by a single individual.

Your assignment solution will consist of a written report that answers the questions, and carries out the tasks, listed below.

1 Background

A fictitious AI company wishes to deploy a command-line interface (CLI) AI assistant as a product. The CLI AI agent is a program that runs locally on the user's computer. Once started, the AI agent takes user input as text and produces its output as text. It also has the ability to run other programs and commands on the user's computer. The AI agent is powered by a Large Language Model (LLM) that runs remotely, and with which the AI agent program communicates: that is how the AI agent does its "thinking": it sends a prompt to the remote LLM and then receives back the LLM's response, which we can think of as the agent's thoughts. Sometimes these thoughts will instruct the agent to execute commands or programs, or to provide a response to the user, etc.

You can think of the AI agent as being somewhat akin to Anthropic's Claude Code, OpenAI's Codex, etc. You are free to draw on your intuitions based on any interactions with those systems you may have had. However, the system description in this document likely departs from the specifics of how those systems operate. Where it does so, consider this assignment specification authoritative.

An example input with the corresponding example output that the model might produce is shown in Figure 1.

```
> Find the keynote file about program analysis

. Search(pattern: "**/*.key")
  - Found 61 files (ctrl+o to expand)

. Search(pattern: "program.analysis")
  - Found 0 files

. The keynote file about program analysis is: Lectures/ProgramAnalysis.key
```

Figure 1: A sample session with the CLI agent.

The way the agent operates is described by the Python pseudocode of Fig. 2. The agent accepts some user input, which is the user's query. It initialises the LLM with a system prompt. The system prompt and the user's query form the initial part of the conversation with the LLM. The agent keeps a record of the conversation so far. Upon querying the LLM, the LLM can respond with one of two types of message: a *ToolInvocation* message allows the LLM to instruct the agent to run a tool on the user's machine, to help the agent to answer the user's query. If the LLM returns such a message, the agent then runs the tool and returns the tool's answer to the LLM. This process repeats until the LLM responds instead with a *UserResponse* message, which is the final answer that the agent returns to the user.

We can see two "Search" tool invocations in Fig. 1 occurred. The actual messages exchanged between the AI agent CLI

```

conversation = [
    SystemPrompt(
        role="You are a coding assistant operating in a local repository.",
        guidance="When answering questions, prefer using tools rather than guessing.",
        tools={...},
    ),
    UserQuery(text=user_input),
]

while True:
    llm_response: LLMMessage = LLM(conversation)
    conversation.append(llm_response)

    if isinstance(llm_response, ToolInvocation):
        tool_result = run_tool(llm_response.tool, llm_response.arguments)
        conversation.append(tool_result)

    elif isinstance(llm_response, UserResponse):
        print(llm_response.text)
        break

```

Figure 2: Python pseudocode showing how the agent answers a single user query.

application and the LLM are depicted in Fig. 3. We can see that the system prompt tells the LLM about what tools are available and how each tool can be used. The LLM uses this information to help it construct useful tool requests for the agent. The system prompt also influences what tools the LLM might choose to use. In the example in Fig. 3 we see that the LLM chose to use the Grep tool unnecessarily, likely because it was influenced by the system prompt. This shows how the conversation that the agent maintains influences the decisions that the LLM makes, including tool invocation requests. As can be seen in Fig. 3 the conversation includes all prior LLM and tool interactions.

The tools available to the agent you can assume are standard ones like being able to search the web, fetch URLs, perform file system operations (search, read, modify, and delete files), etc.

The AI CLI agent product that the company wishes to deploy is called *Codey McCodeface*. The company developing Codey McCodeface expects it to be a highly popular product, being useful not only for CLI tasks (as in the example above) but also coding and other computer use tasks. Expecting Codey McCodeface to be in high demand, but also wishing to leverage its users as unpaid testers to help improve the AI agent’s performance, Codey McCodeface is designed to offer various levels of service:

Free Users can use the AI agent without having to pay, but can only use the agent a fixed number of times per (unspecified) time unit—in other words, Free usage is rate-limited. Rate-limiting is performed by having Free users register with an account against which their usage is tracked. Doing so is designed to ensure that the AI product does not become overwhelmed with requests.

Paid Users can also pay to use the AI agent. Each use of the agent incurs an (unspecified) fee. Paid users provide credit card or other equivalent billing details. At the end of each month, Codey McCodeface charges their billing method to account for their use of the agent during the past month.

Users interact with Codey McCodeface via the CLI application. The CLI application provides a “history” feature, allowing users to save and restore their history of interactions with the agent. Each user session is recorded, including the history of inputs and outputs to the agent in that session. Users can save their current session and then restore it at a later time, allowing them to continue where they left off with the agent (e.g., to build on previous results and interactions with the agent). Each session is assigned a unique name, and sessions are auto-saved.

Sessions are saved on a remote server so that the same user can use Codey McCodeface on different devices and still restore previous sessions.

```

Agent -> LLM
SystemPrompt
role: You are a coding assistant operating in a local repository.
guidance: When answering questions, prefer using tools rather than guessing.
tools:
  Glob
    friendly-name: Search
    description: Find files whose paths match a glob pattern.
    arguments:
      glob-pattern: string
  Grep
    friendly-name: Search
    description: Regex search all files whose paths match a glob pattern.
    arguments:
      regex: string
      glob-pattern: string
      case-insensitive: boolean
  ...

Agent -> LLM
UserQuery
text: Find the keynote file about program analysis

LLM -> Agent
ToolInvocation
thinking: The system prompt suggests locating candidate files first.
tool: Glob
arguments:
  glob-pattern: **/*.key

Agent -> LLM
ToolResponse
tool: Glob
result:
  61 files found
  Lectures/Lecture1-Intro.key
  ...
  Lectures/ProgramAnalysis.key
  ...

LLM -> Agent
ToolInvocation
thinking: The filename ProgramAnalysis.key looks like a strong match. Let's confirm using Grep.
tool: Grep
arguments:
  regex: program.analysis
  case-insensitive: true
  glob-pattern: *.key

Agent -> LLM
ToolResponse
tool: Grep
result:
  No files found

LLM -> Agent
UserResponse
thinking: The Grep search failed. But there is a likely filename in the earlier file list.
text: The keynote file about program analysis is: Lectures/ProgramAnalysis.key

```

Figure 3: The internal protocol between the agent and the LLM for the session in Figure 1.

After submitting a query and receiving a final answer from the agent, which it then displays to the CLI application, the CLI application provides a “thumbs up” and “thumbs down” menu items that the user can choose between to indicate whether the answer that the agent produced was a desirable response to the submitted input (“thumbs up”) or not (“thumbs down”). This allows the Codey McCodeface developers to periodically improve the AI agent by re-training the LLM that powers it, based on this user feedback.

The AI agent is powered by a remote LLM that runs on high-performance backend cloud servers providing GPU compute capability (on which the LLM executes).

There is also a web site that users use to sign-up to the service, create accounts, and so on. However, the agent is accessed only through the CLI application.

The web site and the LLM are hosted by separate third-parties. Behind the scenes, Codey McCodeface makes use also of other third-party providers.

The company that developed Codey McCodeface makes use of a third-party billing provider that does the end-of-month billing for Paid customers. It makes use of a third-party identity provider to implement authentication for its users. It makes use of a third-party Content Distribution Network (CDN) provider that hosts the web site that users use to sign-up to the service and manage their accounts. It makes use of a third-party cloud provider that provides high-performance computing services on which the LLM runs.

Sensitive Information Codey McCodeface and its associated infrastructure processes a range of sensitive information. This includes user input to the agent and the agent’s responses (via the CLI), previous user sessions, billing information, authentication information (e.g., usernames and password hashes or other authentication tokens). It may also process other information (see the note about ambiguity below).

Security Goals In addition, Codey McCodeface should protect not only the data of its users but also other attributes of its users. Specifically, Codey McCodeface has a goal of not *endangering* its users, by avoiding causing them unacceptable physical harm, offence, emotional distress, financial loss, embarrassment, reputational damage, and so on. Codey McCodeface should also minimise the risk that its users will be able use Codey McCodeface to endanger others, including non-users.

2 System overview

The system contains a number of distinct components.

User Browsers and devices Users access Codey McCodeface via its the CLI application, running on their personal computer(s).

Public Web Interface The public web interface runs on a Content Distribution Network (CDN) (e.g. like Cloudflare).

As well as allowing for account sign-up, etc., the web interface also performs user session management. It stores all user session data, including histories, in a database. This database also records how many queries a user submits to the system, which is used for billing purposes for Paid users.

This database stores the histories of CLI agent sessions that users save and restore via the CLI application. These sessions are stored remotely so that users can restore them from different devices. For the purposes of this assignment, the database should be considered part of the Public Web Interface component: it runs on the same CDN servers as the web interface and is administered together with it.

The web interface communicates with the Identity Provider to authenticate users when they log-in. The CLI application also uses the Identity Provider to authenticate the user when they run the CLI application.

The CLI applicatoin communicates with the web interface so that the latter can track Paid users' usage. When a user signs up, they provide their billing information to the Billing Provider via the web interface. At the end of the month, the Billing Provider and web interface communicate to allow the former to calculate the end-of-month bill for each Paid customer, based on that customer's usage. Customer billing information is not stored on the web interface and is instead stored by the Billing Provider.

The machines on which the web interface runs are of course controlled by the CDN; however they are administered by the company that created Codey McCodeface.

AI Agent The LLM that powers Codey McCodeface runs on GPU compute facilities provided by a third-party cloud provider (e.g., like AWS). User input is provided to the LLM from the CLI agent and LLM responses are then sent back to the CLI agent to be displayed to the user via their terminal.

The machines on which the LLM runs are of course controlled by the Cloud Provider; however they are administered by the company that created Codey McCodeface.

System Administrators and Administrator Computers System administrators are staff that work for the company developing Codey McCodeface and who run, maintain and administer the Codey McCodeface system. These staff members administer both the web interface machines (on the CDN) and the machines that run the LLM model (on the cloud provider), even though both kinds of machines are under the control of the CDN and the cloud provider respectively.

The system administrators periodically re-train the LLM that powers Codey McCodeface.

The system administrators access and administer the system using special-purpose computers on the internal company network of the company who built the Codey McCodeface system.

External System: Identity Provider The Identity Provider is an external system run by a large company (e.g., like Google or Okta). When a user signs up for Codey McCodeface, or tries to log-in, the web interface or CLI agent communicates with the Identity Provider to accomplish these tasks. This means that the Codey McCodeface system doesn't need to worry about how the authentication process is implemented. It trusts the Identity Provider to vouch for the authenticity of users who log-in to the system.

External System: Billing Provider At the end of each month, the Web Interface and Billing Provider communicate, allowing the third-party Billing Provider to calculate the monthly bill for each Paid customer.

When a user signs-up for Codey McCodeface, the web interface provides their billing information to the Billing Provider, which stores this information for billing purposes.

3 Your Tasks

1. **[(2 marks)]** Draw a block diagram of the architecture of the system, including its main components and the legitimate channels of communication between them. For each communication channel, label it with the sorts of sensitive information (see Section 1) that flow on that channel. For each component of the system that stores sensitive data, label it with the sorts of sensitive information that it stores.

For each component, describe in no more than a few sentences:

- (a) Who has ultimate control over that component?
- (b) What is its role in the system and how is it intended to interact with the other components?

On your diagram, indicate the trust boundaries that exist within the system. For each trust boundary, describe who ultimately controls the components within that boundary.

Trust boundaries can only exist *between* components (not *within* them, i.e. a single component can live inside only one trust boundary). If you have think you need to draw a trust boundary that passes through a component, this indicates that that component should be split into multiple ones, each of which then resides in a separate trust boundary.

2. [(4 marks)] Use the STRIDE methodology to enumerate potential security threats to the system. For each threat that you identify you should document:

- (a) Who is the potential attacker that might try to exploit this threat?
- (b) What is the security goal that the attack or threat would violate if it were successful?
- (c) How might the attacker exploit this threat?

NOTE ON AMBIGUITY: Importantly, your report should *document and justify any assumptions you make while carrying out your analysis*. The system description provided above is intentionally ambiguous. You might therefore need to make certain assumptions when carrying out your analysis. You should make sure that your assumptions are reasonable, by including with each a brief justification.

When documenting how an attacker might exploit a particular threat, try to be as specific as you can and to draw on past incidents or vulnerabilities to justify your choice.

For example, the threat that an attacker might try to impersonate another user is a bit vague. How might they try to impersonate them and for what purpose? Have such attacks been carried out in the past and if so, how?

For this question we are expecting you to find a range of potential threats, from relatively simple threats to sophisticated ones. There is no fixed number of threats you are aiming to find. Instead, we want to see that you have been *methodical* and *creative*: if you are methodical in how you apply STRIDE, you should be unlikely to miss simple threats. If you are creative and do your research (see next paragraph) you will find some interesting potential threats in this system. As a rough guide: around five is far too few threats. 40 is likely to be excessive (by that point you may be splitting hairs).

To get full marks for this part of the assignment you should expect to have to do some research on past vulnerabilities and attacks on similar systems. Remember from the lectures on safety engineering the importance of learning from past incidents.

If you believe a certain threat might be realistic but cannot find evidence of it being exploited in the past, justify why you think it is realistic and any assumptions you are making when drawing that conclusion. This can include citing research papers in which certain threats have been theorised about.

3. [(2 marks)] Your next task is to think about the impacts of each of the threats you identified. To work this out, for each threat you should think about what are its potential consequences. For each you should briefly describe the potential impacts that could arise if this threat were exploited. When thinking about impacts you should consider impacts on individuals, the various companies involved in this system, plus the government and society (if applicable), with appropriate justification for your reasoning.

Again you should try to refer to documented instances of your threats that have occurred in the past, and the impacts that those attacks had, to justify your reasoning, where such exist.

4. [(2 marks)] Based on the assessment of the severity of each threat, derive a corresponding set of security requirements for the system that would address or mitigate that threat, if that threat can be mitigated. Any threats that you think cannot be reasonably mitigated you should explain why. That might include why certain mitigation methods are not applicable under the assumptions you are making (which you should document).

List for each threat the requirements that are needed to mitigate it. As before, try to be specific. If you think authentication needs to be employed in a certain part of the system, what kind of authentication and why? To work that out it may help to look at your other threats too—perhaps by employing a certain kind of authentication you can mitigate multiple threats at once.

Number each of your security requirements that you derive. That way, if one security requirement helps to address multiple threats, you don't need to repeat it.

Note also that to get full marks in this part of the assignment you should expect to have to do some research on appropriate security mitigations to defend against certain threats.

Hint: this is not a subject about basic authentication methods, or specific kinds of encryption protocols, or other specific defence mechanisms relevant to this system. However, we do expect you to be able to do some research on different kinds of security mitigation methods available for different kinds of threats, to choose the best one under the threats you have identified and the assumptions you have made in your analysis.

As a graduate student, we expect to see real creativity and a desire to push the boundaries of your knowledge. This assignment is designed to be an ideal opportunity to educate yourself on security threats and defences, as relevant to this kind of system.

4 Marking Criteria

There is not a set of right or wrong answers for this assignment. Instead, it is testing your ability to understand and apply the concepts presented in lectures about security and safety engineering.

If you think that some of the requirements are ambiguous, then you should decide on an appropriate interpretation and, very importantly, you should document what your interpretation was. That way, you cannot be penalised for making an assumption that is different to what I or the markers had in mind.

You are also free to discuss the requirements on the LMS, especially where you think they are ambiguous, to help clarify them.

If you are not sure about whether a particular threat is realistic, or how an attacker might exploit a particular threat, or you are not sure what kind of security mitigation to require to defend against it, you are free to email Toby or your tutor to discuss or ask during a consultation session.

Please try to avoid giving away information about your potential threats and security requirements while discussing with other students, especially on the LMS. Ask the subject staff in the first instance.

5 Submission

One of your pair should create a pdf file called `your_username.pdf`, containing your joint answers to the questions. Submit it via the LMS.

6 Communication Rules

You may discuss the questions freely within your pair, and write up your joint answers together. You may also consult any other materials you find on the Internet (or in the library), as long as you give proper references in your report. You may *not* discuss this with anyone other than your project partner, except to clarify the requirements of the assignment. In particular, cross-pair collaboration is not allowed. However, you may ask or answer any question you like on the LMS discussion board—this is up to you. You may share answers or raise interesting questions if you like, for the benefit of all. This allows ideas to be shared but mitigates the (unfair) advantage of having clever friends.

7 Use of Generative AI

You are permitted to use Generative AI to help you brainstorm threats, research similar threats, and to help propose security mitigations and develop security requirements. However, **you must document how you used Generative AI. When doing so include commentary on how useful or otherwise you found Generative AI for this assignment.** Importantly, **you remain responsible for the content of your submission.**

Any errors that the generative AI introduces are therefore *your* errors.

8 Late submissions

Please submit on time. It's much better to submit a not-quite-finished version on time than a perfect version late. 1 mark will be deducted each day (or part thereof) after the submission deadline. If you have a real reason for needing an extension, please ask permission in advance. I will usually ask to see some form of evidence, e.g. a medical certificate.

9 Academic Misconduct

The University Academic Integrity Policy MPF1310 (<https://policy.unimelb.edu.au/MPF1310>) applies to this assignment. What you submit must be your own work, and where you use the work of others it must be appropriately acknowledged (e.g. by citation).

The subject staff take plagiarism very seriously. In the past, we have successfully prosecuted several students that have breached the university policy. Often this results in receiving 0 marks for the assessment, and in some cases, has resulted in failure of the subject.

Quoting from the University's Statement on the use of AI in assignments (<https://academicintegrity.unimelb.edu.au/plagiarism-and-collusion/artificial-intelligence-tools-and-technologies>):

If a student uses artificial intelligence software such as ChatGPT or QuillBot to generate material for assessment that they represent as their own ideas, research and/or analysis, they are NOT submitting their own work. Knowingly having a third party, including artificial intelligence technologies, write or produce any work (paid or unpaid) that a student submits as their own work for assessment is deliberate cheating and is academic misconduct.

If a student uses AI generated material in the preparation of their assessment submission, this must be appropriately acknowledged and cited in accordance with the Assessment and Results Policy (MPF1326) <https://policy.unimelb.edu.au/MPF1326>.